DMSM — Development Method Selection Module

DMSM — Development Method Selection Module

OOF® Origin Open Foundation®

Independent Methodological Authority

Architecture: SLGA™ — Software Lifecycle Governance Architecture

Parent Standard: Software Creation Standard (SCS)

Category: Governance & Enforcement

Subcategory: Software Lifecycle Governance

Type: Core Module

Governed Space: Development Method Selection Governance

Module ID: SLGA-SCS-DMSM-001

Origin ID: OOF.CM.SLGA.02.01.01

Version: 1.0

Status: Canonical · Open Standard

Effective Date: 11.10.2026

Compatibility: OOF® Methodology OS™ · Modular OOF® Architecture Ecosystem

GIE-Compatible: Yes

AI-Readable: Yes

Authority: OOF® — OriginOpen® Foundation

Protection: MIP® — Methodological Intellectual Property

Canonical Language: English (UCL™)

Canonical Definition

The Development Method Selection Module (DMSM) governs how methods used to

create, generate, implement, transform, or assemble software are identified, evaluated, selected, bounded, justified, and reassessed against the applicable software creation objective, requirements, authority, operational conditions, and governance constraints.

DMSM establishes the methodological conditions under which a development

method may be recognized as suitable for a defined software creation activity.

It distinguishes the availability of a development method from its

suitability, the technical capability of a method from its legitimate use, and the selection of a method from authorization to execute the actions that the method requires.

DMSM applies to human-directed development, AI-assisted programming,

autonomous code generation, model-driven development, software synthesis, low-code and no-code creation, software transformation, and hybrid development approaches.

It governs method selection regardless of whether the selection is performed by a human, organization, AI agent, autonomous system, or automated orchestration mechanism.

Its principal output is:

Governed Development Method Selection State

Its foundational distinction is:

Available Development Method ≠ Governed Development Method Selection.

Operational Role

The operational role of DMSM is:

To ensure that the method chosen to create software is appropriate to the governed creation objective, sufficiently justified against relevant requirements and constraints, compatible with the applicable authority and execution boundaries, and subject to reassessment when material conditions change.

DMSM establishes a governed basis for selecting how software will be created.

It does not prescribe a universally preferred development methodology.

It does not determine that a particular programming language, framework, AI model, or development tool is inherently superior.

Instead, it establishes whether a proposed method is appropriate for the specific creation activity and the conditions under which that activity is permitted to occur.

Position in the SCS Internal Governance Flow

The SCS internal governance flow is:

Development Method Selection → Code & Artifact Integrity → Autonomous

Software Creation Control → Software Artifact Composition → Development Traceability

DMSM governs:

Development Method Selection

It is the first Core Module of the Software Creation Standard (SCS).

Its principal upstream references are:

Governed Software Requirements Baseline

from:

Software Requirements Standard (SRS)

and the applicable software creation objective and authority references

established under SCS.

Its principal output is:

Governed Development Method Selection State

That state provides the methodological basis for the remaining SCS Core Modules.

The flow is not a mandatory chronological development sequence.

Development methods may be selected, combined, revised, or reassessed during iterative and autonomous software creation.

Central Operational Question

Which development method is appropriate for the intended software creation activity, under what requirements, authority, execution, and operational conditions may it be used, and what evidence supports recognizing that selection as governed rather than merely technically convenient?

Development Method Selection Governance Object

Every DMSM assessment shall identify the Development Method Selection

Governance Object.

The object represents a proposed or recognized selection of one or more

methods for a defined software creation activity.

It should identify, where applicable:

Selection Identity: The identifiable method-selection decision or

assessment.

Creation Objective: The intended software creation outcome.

Requirements Reference: The applicable governed requirements basis.

Proposed Method: The method being considered.

Method Scope: The creation activities to which the method would apply.

Selection Actor: The human, organization, AI agent, or autonomous system

proposing or evaluating the method.

Authority Reference: The applicable method-selection authority or

delegation basis.

Execution Conditions: The relevant operational and technical conditions.

Constraints: Material requirements, limitations, and prohibitions.

Evaluation Evidence: The information supporting method assessment.

Selection State: The current governance condition of the method selection.

A development method may be selected for one creation activity without being suitable for another.

Method Suitability Is Context-Dependent.

Development Method Identity

DMSM shall identify the method being proposed or selected.

A development method may represent:

• A software engineering approach

• A code-generation mechanism

• A software transformation method

• A model-driven creation process

• An autonomous development procedure

• A hybrid development arrangement

• A defined combination of methods

The method shall be distinguishable from the specific tools used to execute

it.

For example, AI-assisted development is a method category.

A particular AI coding assistant is a possible tool supporting that method.

Likewise, iterative development is a process approach rather than a particular programming language.

Development Method ≠ Development Tool.

Where method identity is insufficiently defined, its suitability cannot be reliably assessed.

Method Origin and Selection Trigger

DMSM shall identify why a particular method is being considered.

Selection triggers may include:

• A new software creation objective

• A recognized requirements baseline

• A change in development capability

• An organizational development policy

• A technical constraint

• A need to modernize existing software

• Availability of an AI-assisted method

• A proposed autonomous development workflow

• A material change in creation conditions

• Failure or insufficiency of an existing method

The selection trigger shall remain distinguishable from the justification

for adopting the method.

The introduction of a new development tool does not automatically justify changing the method used to create software.

Method Availability ≠ Method Necessity.

Creation Objective Alignment

DMSM shall evaluate whether the proposed development method is appropriate for the recognized software creation objective.

The assessment should consider:

• The intended software outcome

• The nature of the required functionality

• The complexity of the creation activity

• Relevant operating conditions

• Material constraints

• Required artifact characteristics

• Necessary development evidence

A method suitable for generating a simple internal application may be inappropriate for creating software that controls safety-critical physical equipment.

The assessment shall not assume that a method is suitable merely because it can produce executable software.

Executable Output Capability ≠ Objective Suitability.

Requirements Alignment

DMSM shall preserve the relationship between method selection and the

applicable:

Governed Software Requirements Baseline

The selected method should support the creation of software within the requirements and constraints established by that baseline.

Relevant considerations may include:

• Functional requirements

• Non-functional requirements

• Interface requirements

• Operational constraints

• Evidence requirements

• Verification expectations

• Traceability requirements

• Relevant external obligations

Where the proposed method cannot adequately support material requirements, the limitation shall be identified.

A method shall not be considered suitable solely because it can implement the most visible functional features.

Functional Implementation Capability ≠ Requirements Coverage.

Method Scope and Applicability

DMSM shall establish the software creation activities to which the selected

method applies.

Method applicability may be limited to:

• A particular software component

• A defined function

• A software subsystem

• A specific development phase

• A transformation activity

• A generation task

• A defined operating environment

• A particular requirements baseline

A method selected for one scope shall not automatically become applicable

to every other creation activity.

Method Selection ≠ Universal Method Applicability.

Where multiple methods are used, their respective scopes shall remain identifiable.

Method Selection Authority

DMSM shall distinguish authority to propose, evaluate, select, approve, and

execute a development method.

These authorities may be distributed among different actors.

A developer may propose an alternative method without possessing authority to approve it.

An AI agent may identify a more efficient generation approach without possessing authority to change the development process.

An organization may authorize the use of AI-assisted coding while restricting autonomous repository modifications.

DMSM shall preserve the applicable authority basis and delegation

conditions.

Method Recommendation ≠ Method Authorization.

Method Selection Authority ≠ Execution Authority.

DMSM does not independently create legal or organizational authority.

Method Selection Criteria

DMSM shall establish criteria appropriate to the creation objective and

governed context.

Possible criteria include:

Requirements Suitability: Ability to support the applicable requirements.

Technical Feasibility: Whether the method can reasonably produce the intended artifacts.

Authority Compatibility: Whether the method can operate within legitimate authority boundaries.

Execution Controllability: Whether material actions can remain sufficiently bounded.

Evidence Capability: Whether relevant creation decisions and actions can be reconstructed.

Artifact Integrity Support: Whether the method can support identifiable and controlled artifact formation.

Verification Support: Whether the method can produce artifacts and evidence suitable for competent downstream assessment.

Operational Context Suitability: Whether the method is appropriate to the intended software environment.

Dependency Implications: Whether the method introduces material

dependency conditions.

Continuity: Whether its outputs and evidence can remain usable when actors, tools, or organizations change.

The criteria shall reflect the governed creation reality.

DMSM does not require a universal numerical scoring model.

Method Feasibility

DMSM shall evaluate whether the proposed method can reasonably support the intended creation activity under the available conditions.

Relevant feasibility dimensions may include:

• Technical capability

• Available expertise

• Required infrastructure

• Tool availability

• Development resources

• Time constraints

• Required interfaces

• Necessary input quality

• Relevant external dependencies

Feasibility assessment shall preserve material assumptions and limitations.

A method may be technically feasible while remaining unauthorized or unsuitable.

Method Feasibility ≠ Method Legitimacy.

Method Authority Compatibility

DMSM shall evaluate whether the proposed method can operate within the applicable authority boundaries.

A method may require:

• Repository modification

• External tool execution

• Access to sensitive data

• Installation of software packages

• Invocation of cloud services

• Autonomous execution

• Modification of configuration

• Creation of new software interfaces

The availability of these capabilities does not establish permission to use them.

Where the method depends on actions outside the recognized authority boundary, the incompatibility shall be identified.

Method Capability ≠ Authorized Method Execution.

Constituent Action Implications

A development method may require a sequence of constituent actions.

DMSM shall identify material action categories that the method reasonably requires.

For example, an autonomous development method may involve:

Requirements Interpretation → Repository Inspection → Tool Selection →

Code Generation → Test Execution → Artifact Modification

The method-selection assessment shall consider whether these actions can remain within applicable permissions.

DMSM does not independently authorize each execution action.

That responsibility remains with the competent creation and autonomous execution governance processes.

Method Selection ≠ Constituent Action Authorization.

Task Authority ≠ Constituent Action Authority.

Method Risk and Constraint Assessment

DMSM shall identify material risks and constraints affecting method

suitability.

Relevant considerations may include:

• Uncontrolled scope expansion

• Inadequate execution boundaries

• Insufficient artifact provenance

• Inability to reconstruct material decisions

• Unverified generation assumptions

• Unacceptable dependency conditions

• Incompatible evidence requirements

• Inadequate intervention capability

• Excessive reliance on unavailable infrastructure

• Material limitations in method competence

DMSM shall preserve the distinction between identifying a risk and

determining its acceptability.

Risk acceptance belongs to the applicable competent governance authority.

Risk Identification ≠ Risk Acceptance.

Human-Directed Development Methods

DMSM applies to software creation methods primarily executed by human

developers.

These may include:

• Manual programming

• Structured software engineering

• Iterative development

• Component-based development

• Model-driven development

• Formal development approaches

• Hybrid organizational processes

Human execution does not exempt a method from governance assessment.

A familiar method may be unsuitable for a particular creation objective.

Human-Directed Development ≠ Automatically Governed Development.

AI-Assisted Development Methods

DMSM applies when AI systems assist human developers.

Relevant method conditions may include:

• The permitted role of AI-generated suggestions

• Human decision authority

• Applicable source-context boundaries

• Generated artifact review conditions

• Evidence preservation

• Material tool-use restrictions

• Requirements alignment

• Relevant uncertainty

An AI-assisted method may be suitable for one class of creation activities while requiring additional controls for another.

AI Assistance ≠ Autonomous Development Authority.

The existence of human review shall not automatically be treated as

evidence that every underlying generation action was appropriately governed.

Autonomous Development Methods

DMSM applies to methods in which AI agents or autonomous systems

independently perform material software creation activities.

The assessment shall consider whether the method can support:

• Identifiable delegated objectives

• Bounded autonomous execution

• Constituent action permission controls

• Material decision reconstruction

• Exception recognition

• Human or competent intervention

• Artifact provenance

• Governance state preservation

A method that can generate software autonomously may remain unsuitable where its execution boundaries cannot be adequately established.

Autonomous Creation Capability ≠ Autonomous Method Suitability.

Detailed autonomous execution governance remains within ASCCM.

Multi-Agent Development Methods

DMSM shall assess methods that distribute creation activities across multiple agents.

Relevant considerations include:

• Task decomposition

• Agent specialization

• Handoff structure

• Authority delegation

• Shared context

• Tool-access boundaries

• Output integration

• Evidence continuity

• Conflict handling

A multi-agent method shall not be treated as suitable solely because it

distributes work efficiently.

Agent Coordination Efficiency ≠ Governance Adequacy.

Where authority and evidence cannot remain sufficiently continuous across agents, the method-selection state shall reflect that limitation.

Hybrid Development Methods

Software creation may combine multiple methods.

Examples include:

• Human design with AI code generation

• AI-generated prototypes followed by human implementation

• Autonomous coding with human review

• Model-driven generation combined with manual modification

• Automated transformation followed by manual integration

DMSM shall identify the relevant method components and their applicability

boundaries.

Hybrid development shall not be treated as a single undifferentiated method

where its constituent approaches create materially different governance conditions.

Hybrid Method Label ≠ Governed Method Integration.

Method Combination and Compatibility

Where multiple methods operate together, DMSM shall assess their material compatibility.

Relevant questions include:

• Do the methods rely on compatible requirements interpretations?

• Can their artifacts be integrated?

• Are their authority boundaries consistent?

• Can material decisions be reconstructed?

• Are their evidence outputs compatible?

• Do their execution conditions create conflicts?

• Can their results be assessed through competent downstream processes?

A combination of individually suitable methods may produce an unsuitable overall creation arrangement.

Individually Suitable Methods ≠ Suitable Method Combination.

Method Evidence Capability

DMSM shall evaluate whether the proposed method can produce or preserve sufficient evidence of material creation activities.

Relevant evidence may include:

• Creation instructions

• Actor participation

• Method decisions

• Tool execution references

• Material inputs

• Artifact generation records

• Review records

• Exception records

• Relevant authority references

A method that produces functional software but cannot preserve material creation evidence may be unsuitable for contexts requiring reconstructable creation histories.

Method Output Capability ≠ Method Evidence Capability.

Method Artifact Integrity Support

DMSM shall evaluate whether the method can support identifiable and sufficiently controlled software artifact formation.

Relevant considerations may include:

• Artifact identity

• Version references

• Origin preservation

• Transformation visibility

• Generated artifact distinguishability

• Integrity evidence

• Relevant composition conditions

DMSM does not independently validate artifact integrity.

That responsibility belongs to:

CRIM — Code & Artifact Integrity Module

DMSM determines whether the method can support the necessary artifact integrity governance.

Integrity Support Capability ≠ Established Artifact Integrity.

Method Traceability Support

DMSM shall assess whether the selected method can support the reconstruction of material relationships between:

• Creation objectives

• Requirements

• Actors

• Method decisions

• Execution activities

• Resulting artifacts

• Material evidence

A method may require additional supporting mechanisms to preserve these relationships.

DMSM does not independently establish the complete development traceability state.

That responsibility belongs to:

DTM — Development Traceability Module

Traceability Support ≠ Governed Development Traceability.

Method Verification and Validation Support

DMSM shall consider whether the selected method can produce artifacts and evidence suitable for competent verification and validation.

The assessment may concern:

• Testability

• Inspection capability

• Artifact observability

• Reproducibility

• Requirements mapping

• Evidence preservation

• Relevant operating conditions

A method that generates executable software may nevertheless make verification difficult or unreliable.

Executable Output ≠ Verifiable Output.

DMSM does not independently determine software quality or validation outcomes.

Method Operational Reality Alignment

DMSM shall assess whether the method-selection decision reflects the

material conditions under which software creation must occur.

Relevant conditions may include:

• Available expertise

• Infrastructure

• Development environment

• Input reliability

• Tool capabilities

• Operational constraints

• Resource limitations

• Organizational control

• Relevant continuity requirements

Method selection shall not rely exclusively on theoretical capability.

Theoretical Method Capability ≠ Operational Method Suitability.

Where the operational context is uncertain, the uncertainty shall remain explicit.

Method Resource and Efficiency Considerations

DMSM may consider resource requirements when evaluating method suitability.

Relevant dimensions may include:

• Development effort

• Computational resources

• Energy consumption

• Tooling costs

• Infrastructure requirements

• Review burden

• Evidence preservation costs

• Maintenance implications

Efficiency may support method selection.

It shall not override fundamental authority, requirements, integrity, or

safety-related governance conditions.

Lower Development Cost ≠ Governed Method Superiority.

Higher Automation ≠ Higher Governance Quality.

Detailed value, computational efficiency, and resource governance remain

within their competent architectures.

Method Dependency and Lock-In Considerations

A development method may depend on:

• A particular AI model

• A proprietary development platform

• A specific toolchain

• A cloud service

• A vendor-controlled generation system

• A specialized programming environment

• An external orchestration framework

DMSM shall identify material dependencies affecting method suitability and

continuity.

The assessment should consider whether resulting artifacts and evidence can remain usable if the method provider becomes unavailable.

Method Availability Today ≠ Method Continuity Tomorrow.

Detailed dependency governance remains within the Software Dependency Standard.

Method Comparison

Where multiple methods are reasonably available, DMSM may compare them against the applicable governance criteria.

Comparison should preserve:

• The creation objective

• Relevant requirements

• Material constraints

• Authority compatibility

• Execution controllability

• Evidence capability

• Artifact integrity support

• Operational feasibility

• Material uncertainty

The comparison shall not assume that the most automated, fastest, cheapest, or newest method is necessarily the most suitable.

Method Ranking ≠ Method Legitimacy.

A method may be rejected despite favorable efficiency metrics where fundamental governance conditions are not satisfied.

Method Selection Justification

DMSM shall preserve a sufficient justification for the selected method.

The justification should explain:

• Why the method was considered

• Which creation objective it supports

• Which requirements and constraints apply

• Which material alternatives were considered, where relevant

• Which limitations were identified

• Which authority conditions apply

• Why the method is considered suitable

• Which conditions remain unresolved

The required depth of justification shall reflect the materiality of the

creation activity.

Documented Selection ≠ Justified Selection.

Method Selection Conditions

A development method may be recognized as suitable only under defined conditions.

Examples include:

• Restricted software scope

• Human review requirements

• Limited tool permissions

• Approved development environments

• Additional artifact integrity controls

• Defined evidence preservation

• Prohibited external actions

• Required intervention capability

• Restricted autonomous execution

DMSM shall preserve these conditions as part of the method-selection state.

Conditional Method Suitability ≠ Unrestricted Method Approval.

Method Selection Governance States

DMSM shall support identifiable method-selection states.

Possible states include:

Proposed: A method has been proposed for evaluation.

Under Evaluation: The method is being assessed against applicable conditions.

Evidence-Limited: Available evidence is insufficient for a reliable determination.

Conditionally Suitable: The method is suitable subject to explicit conditions.

Suitable: The method satisfies the applicable suitability criteria.

Unsuitable: The method does not satisfy material suitability conditions.

Conflicted: Material assessment contradictions remain unresolved.

Authorized for Use: Competent authorization for the defined method use

has been established.

Suspended: Recognition or use has been temporarily interrupted.

Superseded: A later governed selection state has replaced the earlier

state.

Unresolved: Available information does not support a sufficiently determined state.

Suitability and authorization shall remain distinguishable.

A method may be suitable but not authorized.

A previously authorized method may become unsuitable when material conditions change.

Suitable Method ≠ Authorized Method Use.

Method Selection and Execution Handoff

DMSM shall preserve the distinction between selecting a method and

executing it.

The handoff should identify:

• Selected method

• Applicable creation objective

• Requirements reference

• Method scope

• Relevant authority

• Execution conditions

• Material constraints

• Evidence expectations

• Applicable governance state

The receiving creation process shall not interpret method selection as

unlimited permission to perform all technically possible actions.

Method Selection ≠ Execution Authorization.

Method Change and Replacement

A selected development method may need to be modified or replaced.

Possible triggers include:

• Requirements changes

• New technical constraints

• Method performance limitations

• Tool unavailability

• New security conditions

• Insufficient evidence capability

• Changes in authority

• Autonomous execution risks

• Development environment changes

• Discovery of a more suitable alternative

DMSM shall preserve the relationship between the previous method, proposed

replacement, evaluation basis, and resulting selection state.

A method replacement shall not automatically inherit all conditions

associated with the previous method.

Method Replacement ≠ Automatic Governance Continuity.

Method Drift

Method Drift occurs when the method actually used materially differs from the recognized method-selection state.

Examples include:

• An AI-assisted process becoming fully autonomous

• Unapproved tools being introduced

• Human review being removed

• A development pipeline expanding its permissions

• A generation process changing its underlying model or execution

• An implementation method shifting to a materially different technical

mechanism. approach.

DMSM shall identify material drift and support reassessment.

Selected Method ≠ Actual Method by default.

The actual execution evidence remains relevant to determining whether the

selected method was followed.

Method Reassessment

DMSM shall support reassessment when material conditions change.

Reassessment triggers may include:

• New requirements

• Changes in software scope

• New operational evidence

• Changes in development actors

• New autonomous capabilities

• Changes in tool permissions

• Method-provider changes

• Material dependency changes

• New evidence-integrity concerns

• Changes in the required verification approach

Reassessment shall preserve previous method-selection states where necessary for historical reconstruction.

Historical Method Suitability ≠ Permanent Method Suitability.

Governed Development Method Selection State

The principal output of DMSM is: Governed Development Method Selection State

The state shall preserve sufficient information to identify:

Selection Governance Object: The identifiable method-selection assessment.

Selection Identity: The relevant selection reference.

Creation Objective: The software creation outcome.

Requirements Reference: The applicable governed requirements basis.

Proposed Method: The method evaluated.

Selected Method: The method recognized for the defined context.

Method Scope: The creation activities to which the method applies.

Selection Origin: The trigger or source of the selection.

Selection Actor: The actor proposing or evaluating the method.

Authority Basis: Applicable selection and use authorization references.

Evaluation Criteria: The material criteria applied.

Method Feasibility: The assessed feasibility conditions.

Requirements Alignment: The relationship to applicable requirements.

Authority Compatibility: The method's compatibility with recognized

authority boundaries.

Constituent Action Implications: Material execution actions and

permission dependencies.

Operational Conditions: Relevant execution context.

Evidence Capability: The method's capacity to preserve creation evidence.

Artifact Integrity Support: Relevant integrity-support conditions.

Traceability Support: Relevant development traceability conditions.

Material Dependencies: Relevant tools, infrastructure, and provider

dependencies.

Selection Justification: The basis for recognizing the method.

Selection Conditions: Explicit limitations or required controls.

Uncertainty: Material unresolved assumptions or evidence gaps.

Selection Governance State: The current suitability and authorization

condition.

Continuity: Historical selection, replacement, and reassessment

relationships.

Creation Handoff: Information required by the remaining SCS governance

processes.

The state shall not independently imply authorization of every constituent

execution action, artifact integrity, software quality, or release readiness.

Minimum Implementation Framework

A conforming implementation of DMSM must provide the following six functional capabilities.

• Development Method Object and Context Governance

Capability to identify the method-selection object, creation objective,

applicable requirements, proposed methods, selection origin, and relevant creation context.

The implementation must distinguish method identity from tool identity and preserve method applicability boundaries.

• Method Suitability and Requirements Alignment Governance

Capability to evaluate proposed methods against requirements, creation

objectives, technical feasibility, operational conditions, and material constraints.

The implementation must not infer suitability solely from executable output capability.

• Method Authority and Execution Boundary Governance

Capability to identify selection authority, method-use authorization,

constituent action implications, and relevant execution constraints.

The implementation must distinguish method selection from authorization of material execution actions.

• Method Evidence, Integrity Support, and Continuity Governance

Capability to assess whether the method can support artifact identity, provenance, creation evidence, development traceability, and continuity across relevant changes.

The implementation must not represent support capabilities as completed integrity or traceability determinations.

• Method Selection State, Change, and Drift Governance

Capability to preserve selection justifications, conditions, governance

states, method replacements, material drift, and reassessment history.

The implementation must distinguish previously recognized method suitability from current suitability.

• Governed Method Selection State and Creation Handoff Governance

Capability to produce a Governed Development Method Selection State with

sufficient information for the remaining SCS Core Modules and competent creation execution processes.

The implementation must preserve unresolved limitations and explicit selection conditions during handoff.

GIE™ Compatibility

DMSM is compatible with GIE™ as a downstream methodological analysis and execution layer.

GIE™ may support:

• Development method discovery

• Method classification

• Creation objective analysis

• Requirements-to-method alignment

• Method feasibility assessment

• Authority compatibility analysis

• Constituent action identification

• Method comparison

• Evidence capability assessment

• Artifact integrity support assessment

• Traceability support assessment

• Method dependency analysis

• Selection condition identification

• Method drift detection

• Method reassessment

• Governed selection state representation

GIE™ may generate method recommendations or comparative evaluations.

Such recommendations shall remain distinguishable from competent method

selection and authorization.

GIE™ must not redefine DMSM governance requirements to fit available software capabilities or automation constraints.

OOF® logic adapts to the governed reality. The engine adapts to OOF®

logic.

Governance Logic

The canonical DMSM governance logic is:

Receive Governed Software Requirements Baseline

↓

Identify Software Creation Objective

↓

Identify Development Method Selection Governance Object

↓

Identify Proposed Development Methods

↓

Define Method Scope and Applicability

↓

Identify Method Selection Origin and Trigger

↓

Identify Selection Authority and Delegation Conditions

↓

Establish Method Evaluation Criteria

↓

Assess Creation Objective Alignment

↓

Assess Requirements Alignment

↓

Assess Method Feasibility

↓

Assess Authority and Execution Boundary Compatibility

↓

Identify Constituent Action Implications

↓

Assess Operational Conditions and Constraints

↓

Assess Evidence Capability

↓

Assess Artifact Integrity Support

↓

Assess Development Traceability Support

↓

Assess Method Dependencies and Continuity

↓

Compare Material Alternatives Where Applicable

↓

Evaluate Selection Justification

↓

Identify Required Selection Conditions

↓

Determine Method Suitability State

↓

Determine Applicable Method-Use Authorization State

↓

Establish Governed Development Method Selection State

↓

Handoff to Remaining SCS Core Modules

↓

Reassess When Material Method or Creation Conditions Change

The principal output is:

Governed Development Method Selection State

Governance Outputs

DMSM may produce:

Development Method Selection Governance Object

Development Method Identity Record

Method Selection Origin Record

Creation Objective Alignment Assessment

Requirements-to-Method Alignment Assessment

Method Scope and Applicability Record

Method Selection Authority Reference

Method Evaluation Criteria

Method Feasibility Assessment

Method Authority Compatibility Assessment

Constituent Action Implication Record

Method Risk and Constraint Assessment

Human-Directed Method Assessment

AI-Assisted Method Assessment

Autonomous Method Assessment

Multi-Agent Method Assessment

Hybrid Method Assessment

Method Combination Compatibility Assessment

Method Evidence Capability Assessment

Method Artifact Integrity Support Assessment

Method Traceability Support Assessment

Method Verification Support Assessment

Method Operational Reality Alignment Assessment

Method Resource and Efficiency Assessment

Method Dependency and Continuity Assessment

Method Comparison Record

Method Selection Justification

Method Selection Conditions

Method Selection Governance State

Method Execution Handoff Record

Method Replacement Record

Method Drift Record

Method Reassessment Record

The principal output is:

Governed Development Method Selection State

Critical Distinctions

DMSM preserves the following distinctions:

Available Development Method ≠ Governed Development Method Selection

Method Suitability Is Context-Dependent

Development Method ≠ Development Tool

Method Availability ≠ Method Necessity

Executable Output Capability ≠ Objective Suitability

Functional Implementation Capability ≠ Requirements Coverage

Method Selection ≠ Universal Method Applicability

Method Recommendation ≠ Method Authorization

Method Selection Authority ≠ Execution Authority

Method Feasibility ≠ Method Legitimacy

Method Capability ≠ Authorized Method Execution

Method Selection ≠ Constituent Action Authorization

Task Authority ≠ Constituent Action Authority

Risk Identification ≠ Risk Acceptance

Human-Directed Development ≠ Automatically Governed Development

AI Assistance ≠ Autonomous Development Authority

Autonomous Creation Capability ≠ Autonomous Method Suitability

Agent Coordination Efficiency ≠ Governance Adequacy

Hybrid Method Label ≠ Governed Method Integration

Individually Suitable Methods ≠ Suitable Method Combination

Method Output Capability ≠ Method Evidence Capability

Integrity Support Capability ≠ Established Artifact Integrity

Traceability Support ≠ Governed Development Traceability

Executable Output ≠ Verifiable Output

Theoretical Method Capability ≠ Operational Method Suitability

Lower Development Cost ≠ Governed Method Superiority

Higher Automation ≠ Higher Governance Quality

Method Availability Today ≠ Method Continuity Tomorrow

Method Ranking ≠ Method Legitimacy

Documented Selection ≠ Justified Selection

Conditional Method Suitability ≠ Unrestricted Method Approval

Suitable Method ≠ Authorized Method Use

Method Selection ≠ Execution Authorization

Method Replacement ≠ Automatic Governance Continuity

Selected Method ≠ Actual Method by default

Historical Method Suitability ≠ Permanent Method Suitability

Cross-Architecture Boundary

DMSM governs development method selection within SCS and SLGA™.

It does not independently govern every action performed through the selected method.

Within SCS:

DMSM — Development Method Selection Module governs method identification,

evaluation, selection, suitability, and reassessment.

CRIM — Code & Artifact Integrity Module governs software artifact

identity, integrity, origin, and controlled transformation.

ASCCM — Autonomous Software Creation Control Module governs autonomous

creation execution and its authority boundaries.

SACM — Software Artifact Composition Module governs the assembly and

integration of creation artifacts.

DTM — Development Traceability Module governs the reconstructable

relationships between creation objectives, actors, actions, and artifacts.

Within SLGA™:

Software Requirements Standard (SRS) establishes the governed

requirements basis.

Software Dependency Standard (SDS) governs software dependencies.

Software Configuration Standard (SCFS) governs software configuration.

Software Quality Standard (SQS) governs software quality.

Software Security Standard (SSS) governs software security.

Software Release Standard (SRLS) governs software release.

Software Deployment Standard (SDPS) governs software deployment.

Software Change Management Standard (SCMS) governs broader lifecycle

changes.

Software Retirement Standard (SRTS) governs software retirement.

Software Lifecycle Control Standard (SLCS) governs cross-cutting

lifecycle control.

Relevant interoperating OOF® architectures may include:

GOA™ — Governance Architecture: Authority and delegation boundaries.

AGA™ — Accountability Governance Architecture: Responsibility and

accountability conditions.

ASGA™ — Autonomous Systems Governance Architecture: Autonomous execution

and control boundaries.

AGI™ — Agent Governance Infrastructure: Agent authority, permission,

coordination, and tool-action governance.

ORA™ — Operational Reality Architecture: Competent operational reality

assessment.

VEGA™ — Value Execution Governance Architecture: Relevant computational,

resource, and value-efficiency governance.

DMSM may consume relevant governed states from these domains.

It shall not redefine their competent governance responsibilities.

Canonical Boundary

DMSM begins when a software creation activity requires the identification, evaluation, selection, or reassessment of a development method.

DMSM governs:

• Development method selection objects

• Development method identity

• Method selection origins

• Creation objective alignment

• Requirements alignment

• Method scope and applicability

• Method selection authority

• Method evaluation criteria

• Method feasibility

• Authority compatibility

• Constituent action implications

• Method risks and constraints

• Human-directed development methods

• AI-assisted development methods

• Autonomous development methods

• Multi-agent development methods

• Hybrid development methods

• Method combination compatibility

• Method evidence capability

• Artifact integrity support

• Development traceability support

• Verification and validation support

• Operational reality alignment

• Resource and efficiency considerations

• Method dependency and continuity

• Method comparison

• Method selection justification

• Method selection conditions

• Method selection governance states

• Method execution handoff

• Method replacement

• Method drift

• Method reassessment

DMSM does not independently determine:

• General legal or organizational authority

• Software requirements legitimacy

• Authorization of every constituent execution action

• Complete software artifact integrity

• Autonomous execution conformance

• Software artifact composition validity

• Complete development traceability

• Software quality conformance

• Software security conformance

• Release readiness

• Deployment authorization

DMSM ends when the relevant method-selection assessment has been represented as a:

Governed Development Method Selection State

with sufficient method identity, requirements alignment, suitability

evidence, authority references, execution conditions, uncertainty, and continuity to support competent software creation governance.

Use Case 1 — An Organization Selects Autonomous Development Because It

Is Faster

An organization wants to create a new internal software application.

Its development team proposes using a fully autonomous coding agent because

the agent can generate the application faster than the existing development process.

The agent can:

• Interpret requirements

• Generate code

• Install packages

• Execute tests

• Modify repository files

• Coordinate additional agents

The method appears technically feasible.

However, the organization has not established whether the agent's

autonomous tool permissions are compatible with the application's data-access and development boundaries.

DMSM evaluates the proposed method against:

Creation Objective → Requirements → Method Capability → Authority

Compatibility → Execution Conditions → Evidence Capability → Suitability

The assessment identifies that the method may be technically feasible but requires additional permission boundaries and execution controls.

The method may receive the state:

Conditionally Suitable

rather than unrestricted approval.

Method Feasibility ≠ Method Legitimacy.

Autonomous Creation Capability ≠ Autonomous Method Suitability.

Method Selection ≠ Constituent Action Authorization.

DMSM prevents development speed from becoming a substitute for method

suitability.

Use Case 2 — A Development Method Produces Working Code but Cannot

Preserve Creation Evidence

A software supplier proposes using a proprietary AI generation platform to

create a critical software component.

The platform produces functional code and executable artifacts.

However, the supplier cannot sufficiently preserve:

• Material generation inputs

• Relevant model or tool references

• Transformation history

• Actor and execution relationships

• Evidence of material creation decisions

• Reconstructable artifact provenance

The method is capable of producing the required software functionality.

Its evidence capability is insufficient for the receiving organization's governed creation requirements.

DMSM identifies the difference between:

Technical Creation Capability

and:

Governance-Suitable Creation Method

The method may be classified as:

Evidence-Limited

or:

Unsuitable

for the defined creation context unless the evidence limitations can be adequately addressed.

Method Output Capability ≠ Method Evidence Capability.

Executable Output ≠ Governed Creation Evidence.

DMSM prevents a technically effective method from being selected where its creation process cannot be sufficiently reconstructed.

Canonical Closing Statement

Software development methods are evolving rapidly.

Human programming, AI-assisted coding, autonomous agents, software

synthesis, model-driven generation, and hybrid development systems provide increasingly diverse ways to create software.

But the availability of a method does not establish its suitability.

A method may generate working software while exceeding permitted execution boundaries.

It may produce high-quality artifacts while failing to preserve their creation history.

It may reduce development costs while introducing unacceptable dependencies.

And it may enable autonomous execution without providing sufficient control over the actions required to complete the objective.

Available Development Method ≠ Governed Development Method Selection.

Method Feasibility ≠ Method Legitimacy.

Method Selection ≠ Constituent Action Authorization.

Method Output Capability ≠ Method Evidence Capability.

Suitable Method ≠ Authorized Method Use.

DMSM establishes the methodological conditions under which development

methods are evaluated against the actual creation objective, requirements, authority boundaries, operational conditions, and evidence obligations.

It does not privilege human development over autonomous development.

It does not privilege traditional methods over emerging methods.

It requires every method to remain appropriate to the governed reality in

which it is used.

DMSM transforms development method selection from a preference for

available technical capabilities into a governed creation decision — ensuring that the method used to produce software is justified by its suitability, legitimate execution conditions, and ability to preserve the evidence required for accountable software creation.